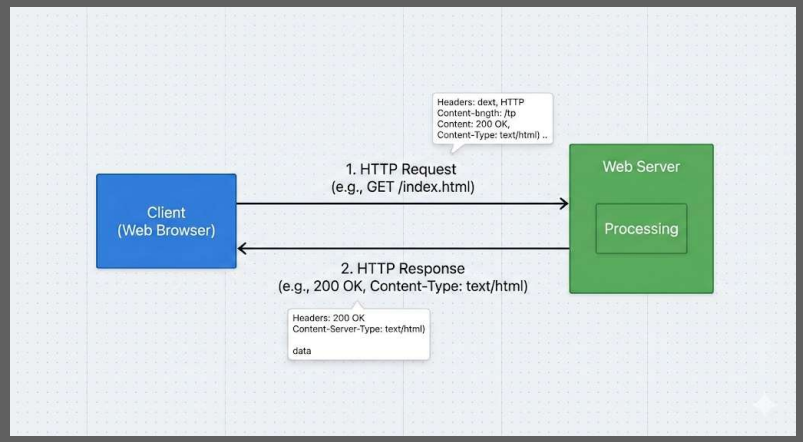
INTRODUCTION TO HTTP SERVICES, WEB SERVERS AND WEB SECURITY

A System Administrator's Guide

WHAT IS THE HTTP SERVICE?

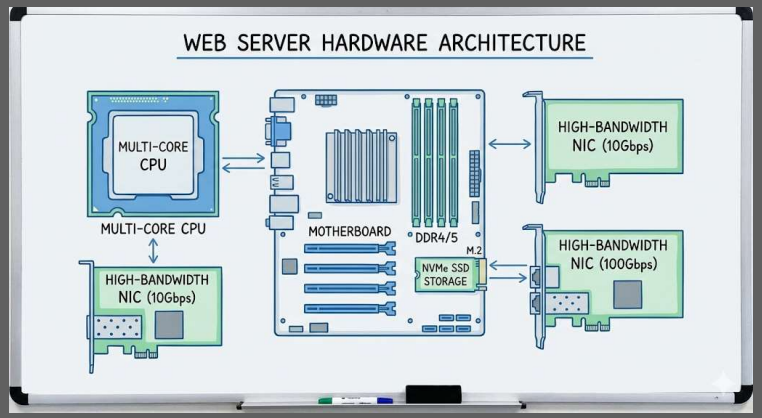
- Hypertext Transfer Protocol (HTTP) is the foundation of data communication on the World Wide Web
- It operates on a Client-Server model
  - A client (like a web browser) sends an HTTP Request, and the
  - Web server return an HTTP Response.



- Ports used:
  - Port 80 (HTTP - unencrypted)
  - Port 443 (HTTPS - encrypted via SSL/TLS)
- HTTP is a stateless protocol:
  - Each request is independent; the server does not keep data between two requests without external mechanisms (like cookies or sessions)

WEB SERVER HARDWARE

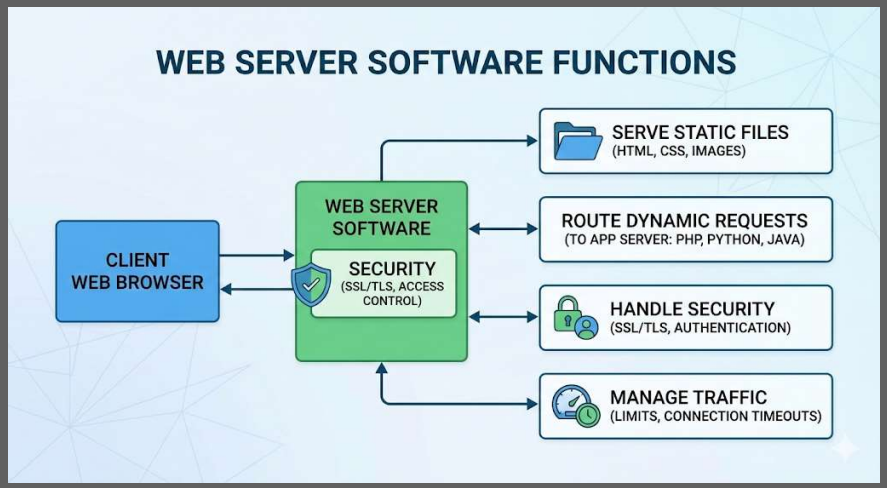
- Overview: While any computer can act as a web server, production environments require specialized hardware optimized for reliabilitylity and speed.
- Key Hardware Components:
  - CPU: Multi-core processors handle concurrent requests and encryption/decryption (SSL/TLS) efficiently.
  - RAM: Crucial for caching frequently accessed data. Insufficient RAM leads to disk swapping, severely degrading performance.
  - Storage: Fast NVMe/SSDs for quick data retrieval of web assets and databases.
  - Network Interface Cards (NICs): High-bandwidth network adapters (1Gbps, 10Gbps, or higher) to handle heavy traffic loads.
  - Redundancy: RAID arrays for storage and dual power supplies to prevent single points of failure.



WEB SERVER SOFTWARE

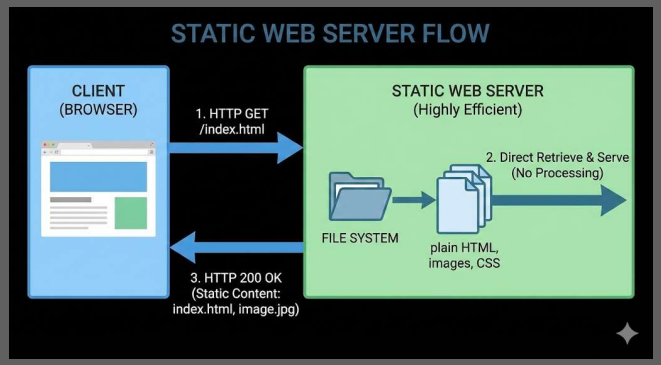
- The role of the web server software is that listens on network ports (80/443), intercepts HTTP requests, and decides how to respond

- Functions:
  - Serving static files (HTML, CSS, Images).
  - Routing requests to application servers for dynamic content (PHP, Python, Java).
  - Handling security (SSL/TLS termination, access control).
  - Managing traffic limits and connection timeouts.

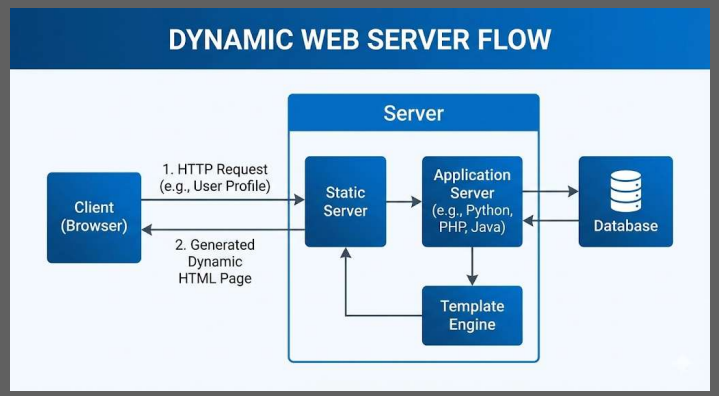


TYPES OF WEB SERVERS

- Static Web Servers
  - Serve files exactly as they are stored (e.g., serving plain HTML and images)
  - Highly efficient

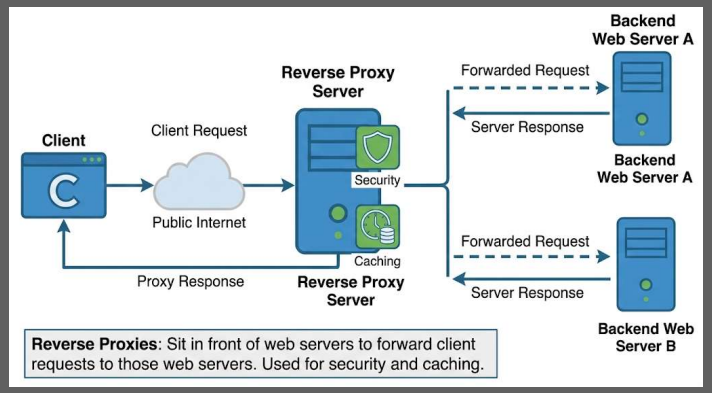


- Dynamic Web Servers
  - Combine a static server with extra software (like an application server and database) to update hosted files before sending them to the browser

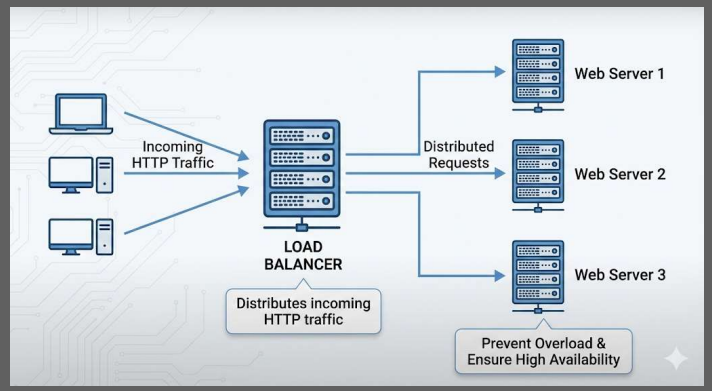


TYPES OF WEB SERVERS

- Reverse Proxies
  - Sit in front of web servers to forward client requests to those web servers. Used for security and caching



- Load Balancers
  - Distribute incoming HTTP traffic across multiple web servers to prevent overload and ensure high availability



WEB SERVERS SUPPORTED ON UBUNTU LINUX

- Ubuntu Linux is one of the most popular OS choices for web hosting
- Common web server software includes:
  - Apache HTTP Server: The historically dominant, highly configurable server
  - Nginx (Engine-X): Built for maximum performance, high concurrency, and low memory usage
  - Lighttpd: Optimized for speed-critical environments with very low memory footprints
  - Caddy: A modern web server known for automatic HTTPS configuration by default
  - Node.js / Tomcat: Application-specific servers for running JavaScript or Java web applications

THE TITANS - APACHE VS. NGINX

Feature	Apache HTTP Server	Nginx
Architecture	Process/Thread-driven (creates a new thread/process for each request).	Event-driven, asynchronous (one worker process handles thousands of requests).
Performance	Excellent for dynamic content, but consumes more RAM under heavy load.	Superior for serving static content and handling concurrent connections.
Configuration	Distributed (.htaccess files allow per-directory configuration).	Centralized (does not support .htaccess, making it slightly faster).
Best Use Case	Shared hosting, complex configurations, legacy apps.	High-traffic sites, reverse proxying, load balancing.

DEPLOYMENT TECHNIQUES FOR UBUNTU LINUX

- Sysadmins have several ways to deploy web servers on Ubuntu:
- Package Managers (APT)
  - The easiest and most common method. Installs stable, tested binaries maintained by the Ubuntu team.
- Source Compilation
  - Downloading the source code and compiling it. Used when you need specific modules or the absolute latest version not yet in APT
- Containerization (Docker)
  - Running the web server in an isolated container
  - Great for microservices, easy teardown, and version control

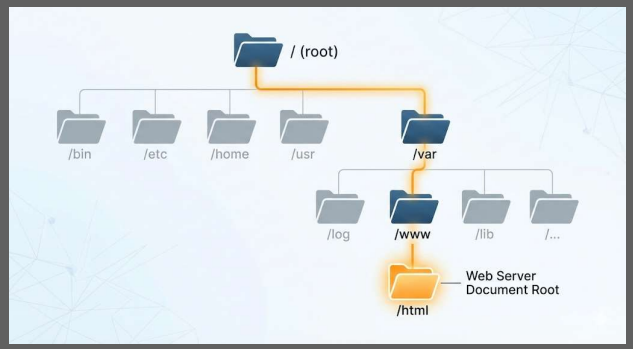
INSTALLATION INSTRUCTIONS (APT METHOD)

- Here is the standard workflow for a system administrator deploying a web server on Ubuntu via the terminal:

- 1. Update System Repositories:
  - sudo apt update && sudo apt upgrade -y
- 2. Install the Web Server (Choose One):
  - For Apache: sudo apt install apache2 -y
  - For Nginx: sudo apt install nginx -y
- 3. Manage the Service (Systemd):
  - Ensure it starts on boot: sudo systemctl enable apache2 (or nginx)
  - Check status: sudo systemctl status apache2
- 4. Configure the Firewall (UFW):
  - Allow HTTP/HTTPS traffic: sudo ufw allow 'Nginx Full' (or 'Apache Full')

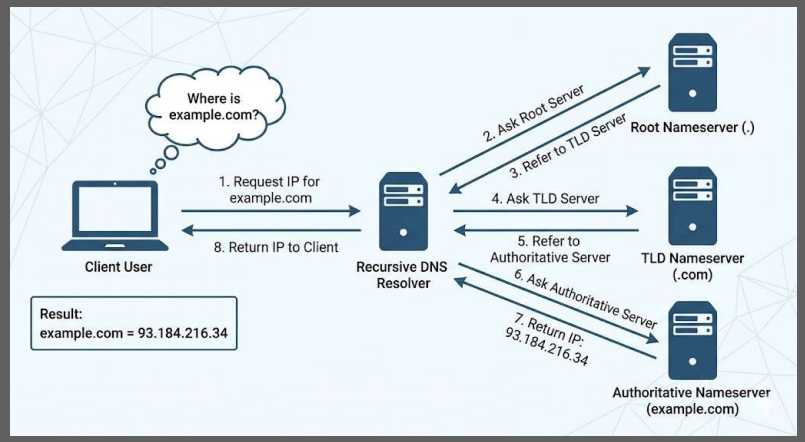
WHERE DO WEB PAGES GO? (THE DOCUMENT ROOT)

- Once the web server is installed, it needs to know where to find the files it should serve to clients
- The Document Root: This is the designated directory on the server's file system where the web server looks for website files
- The Ubuntu Default: Immediately after installing Apache/Nginx via APT, the system creates the default document root at /var/www/html
- The Welcome Page: The installation places a placeholder index.html file in this directory
- Adding Your Pages: To serve your own content, replace that default file with your own index.html, CSS, and media files
- Quick Command-Line Test: Overwrite the default page instantly to test your setup:
  - echo "<h1>Hello Class! The server is running.</h1>" | sudo tee /var/www/html/index.html



PUBLISHING A WEB SITE (THE SYSADMIN WORKFLOW)

- Beyond the default document root, publishing a production site involves connecting a domain to the server and deploying the content securely.
- 1. Domain Name Resolution (DNS): Configure A Records to point the domain to the server's public IP address
- 2. Content Deployment: Transfer files to the server using tools like Secure Copy Protocol (scp), SFTP, or Git
- 3. Custom Document Roots: Create a new directory for each specific site instead of using the default (e.g., /var/www/example.com)
- 4. File Permissions: Set proper ownership (chown) and permissions (chmod) so the www-data user can read files securely
- 5. Virtual Hosts / Server Blocks: Configure the server to listen for specific domain names and route traffic to the correct document root

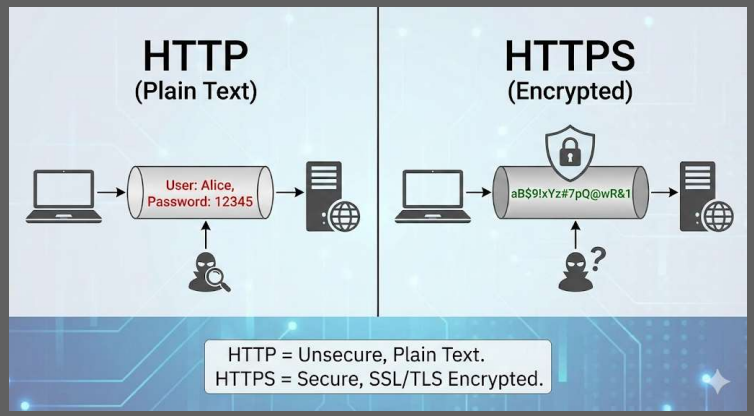


INTRODUCTION TO WEB SECURITY

- What is Web Security?
  - The practice of protecting websites, web applications, and web servers from cyber threats, unauthorized access, and data breaches
- The Purpose - To maintain the CIA Triad:
  - Confidentiality: Keeping user data (passwords, PII) private
  - Integrity: Ensuring the website code and database haven't been maliciously altered
  - Availability: Ensuring the server remains online and accessible to legitimate users
- Importance for System Administration!
  - As the maintainer of the infrastructure, the sysadmin is the first line of defense against data theft and server hijacking

COMMON WEB SECURITY VULNERABILITIES (WEAKNESSES)

- System administrators must avoid these common configuration pitfalls that leave servers exposed:
- 1. Using HTTP vs. HTTPS: Transmitting data via HTTP means everything is sent in plain text, making it trivial for attackers to intercept
- 2. Security Misconfigurations: Leaving default administrative settings unchanged or exposing sensitive directory listings
- 3. No Role-Based Accounts (Broken Access Control): Running the service as the root user, or failing to implement Role-Based Access Control (RBAC)
- 4. Lack of Rate Limiting: Allowing an unlimited number of requests from a single IP address
- 5. No Backups & Disaster Recovery: Treating the live server as the only copy of the data

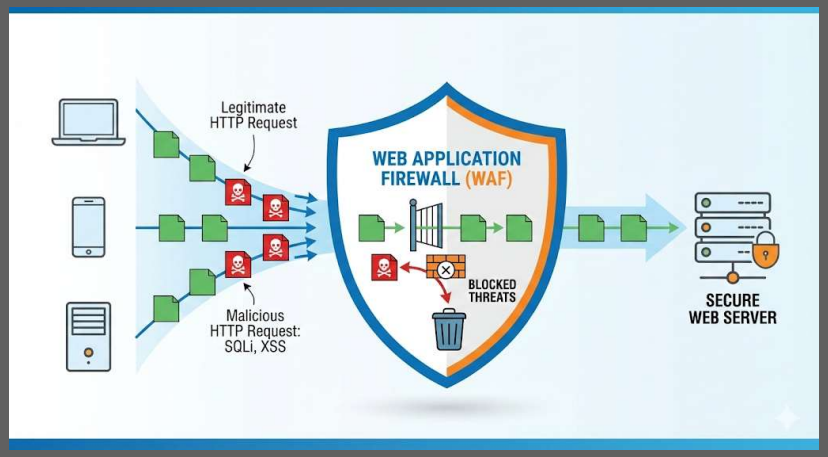


COMMON WEB SECURITY ATTACKS (EXPLOITS)

- A vulnerability is a weakness; an attack is how malicious actors exploit it
- Man-in-the-Middle (MITM): Exploiting HTTP to eavesdrop on or alter unencrypted traffic passing between the client and the server
- Brute Force Attacks: Exploiting a lack of rate limiting to programmatically guess usernames and passwords thousands of times a minute
- DDoS (Distributed Denial of Service): Overwhelming the web server's resources (CPU/RAM/Bandwidth) with flood traffic
- SQL Injection (SQLi) & Cross-Site Scripting (XSS): Injecting malicious code into web forms or pages to trick the database or steal session cookies
- Ransomware & Data Destruction: Exploiting access vulnerabilities to encrypt or delete web directories and databases

THE WEB APPLICATION FIREWALL (WAF)

- Network firewalls (like UFW) block IPs and ports. A WAF inspects the actual web traffic to block malicious payloads
- What is a WAF?
  - A specialized security tool that sits between the internet and your web server, analyzing every HTTP/HTTPS request at Layer 7
- Its Use in Web Security:
  - Pattern Matching: It uses a set of rules to identify and block common attack signatures
  - Stops SQLi & XSS: It instantly detects and drops malicious SQL commands or JavaScript before reaching the database
  - Virtual Patching: Allows sysadmins to instantly block exploits for newly discovered vulnerabilities while developers patch the code

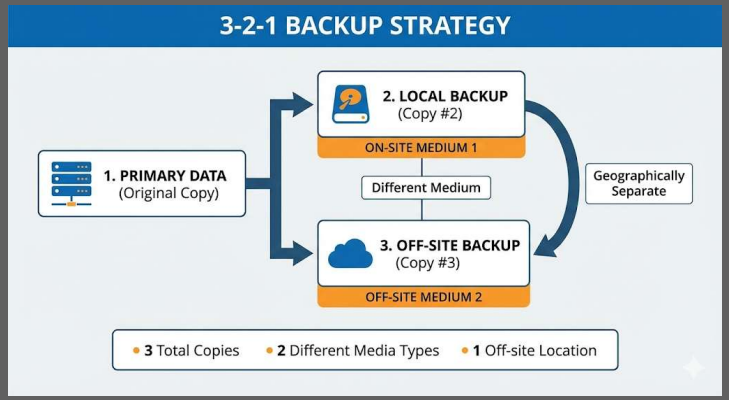


OPEN-SOURCE WAF SOLUTIONS

- System administrators have access to powerful, free, open-source WAF tools to secure their environments
- ModSecurity: The industry standard open-source WAF. Supports Apache, Nginx, and IIS. Provides deep request/response inspection
- The OWASP Core Rule Set (CRS): A set of generic attack detection rules designed for ModSecurity to protect against the OWASP Top 10 vulnerabilities
- NAXSI (Nginx Anti XSS & SQL Injection): A lightweight WAF built for Nginx. Uses a "default deny" approach instead of massive signature lists
- Coraza: A modern, highly performant WAF written in Go. Acts as a drop-in replacement for ModSecurity and natively supports the OWASP CRS

WEB SECURITY BEST PRACTICES (SYSADMIN FOCUS)

- How System Administrators lock down the web environment:
- 1. Enforce HTTPS & Redirect HTTP: Use SSL/TLS certificates (e.g., Let's Encrypt) to encrypt all data in transit
- 2. Implement Rate Limiting: Use tools like fail2ban to throttle excessive requests and block malicious IPs
- 3. Principle of Least Privilege: Ensure the web server runs as a restricted user (like www-data)
- 4. Harden Server Configurations: Disable directory browsing and hide server version signatures
- 5. Deploy Layered Firewalls: Use UFW at the OS level and configure an open-source WAF (like ModSecurity) for Layer 7
- 6. Implement Robust Backups: Follow the 3-2-1 rule (3 copies, 2 mediums, 1 off-site) to ensure data recovery



SUMMARY

- The Foundation: HTTP is the stateless protocol powering the web, relying on robust hardware and specialized software
- The Environment: Ubuntu Linux provides a powerful platform for hosting, with Apache and Nginx as standard choices
- The SysAdmin Workflow: Core duties involve installing software, managing document roots, configuring virtual hosts, and manipulating DNS
- Web Security Risks: Servers face constant threats from vulnerabilities (misconfigurations, HTTP, no backups) and active exploits (SQLi, XSS, DDoS)
- The Defense Posture: Defending the server requires layered security: enforcing HTTPS, rate limiting, deploying a WAF, and maintaining reliable off-site backups

INTRODUCTION TO HTTP SERVICES, WEB SERVERS AND WEB SECURITY

A System Administrator's Guide